ACCESS CONTROL AND AUTHENTICATION

TASK: 1

Aim: Create a mock Zomato admin login form that grants access only when both the username and password match the hardcoded credentials.

Answer:

The following HTML, CSS and JavaScript create a simple mock login form. The form displays “Access Granted” only when both entered values match the sample admin credentials.

```
<!DOCTYPE html>
<html>
<head>
  <title>Mock Zomato Admin Login</title>
</head>
<body>
  <h2>Zomato Admin Panel</h2>

  <form onsubmit="checkLogin(event)">
    <label>Username:</label>
    <input type="text" id="username" required><br><br>

    <label>Password:</label>
    <input type="password" id="password" required><br><br>

    <button type="submit">Login</button>
  </form>

  <p id="message"></p>

  <script>
    function checkLogin(event) {
      event.preventDefault();

      // Demo credentials only; do not use hardcoded credentials in real apps.
      const adminUser = "admin";
      const adminPassword = "Zomato@123";

      const username = document.getElementById("username").value;
      const password = document.getElementById("password").value;
      const message = document.getElementById("message");

      if (username === adminUser && password === adminPassword) {
        message.textContent = "Access Granted";
      } else {
        message.textContent = "Access Denied: Invalid credentials";
      }
    }
```

```
  </script>
</body>
</html>
```

Sample credentials for testing:
Username: admin Password: Zomato@123

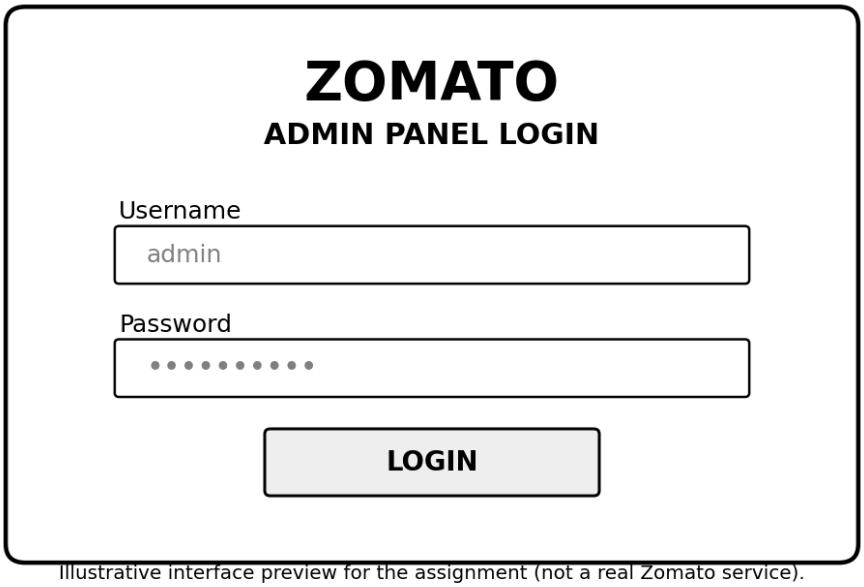


Figure 1: Illustrative preview of the mock admin login form. It is a design preview, not a live Zomato login screenshot.

Result: The mock form grants access only when both the username and password are correct.

TASK: 2

Aim: Write and test a Python role-based access control function.

Answer:

Role-Based Access Control (RBAC) assigns permissions according to a user's role. The dictionary below stores the resources each role is allowed to access.

```
# Map each role to its permitted resources
role_permissions = {
    "customer": {"order_history"},
    "restaurant_owner": {"menu_edit", "order_history"},
    "admin": {"menu_edit", "order_history", "admin_dashboard"}
}

def check_access(user_role, resource):
    allowed_resources = role_permissions.get(user_role, set())
    return resource in allowed_resources

# Test the function
tests = [
    ("customer", "order_history"),
    ("customer", "menu_edit"),
    ("restaurant_owner", "menu_edit"),
    ("restaurant_owner", "admin_dashboard"),
    ("admin", "admin_dashboard")
]

for role, resource in tests:
    print(role, resource, check_access(role, resource))
```

Expected output:

```
customer order_history True
customer menu_edit False
restaurant_owner menu_edit True
restaurant_owner admin_dashboard False
admin admin_dashboard True
```

SAMPLE ROLE-BASED ACCESS CONTROL MATRIX

Role	menu_edit	order_history	admin_dashboard
customer	No	Yes	No
restaurant_owner	Yes	Yes	No
admin	Yes	Yes	Yes

Figure 2: Example role-permission matrix used by the function.

Result: The function returns True when the role has permission for the requested resource; otherwise, it returns False.

TASK: 3

Aim: Simulate two-factor authentication using a password and a six-digit one-time password (OTP).

Answer:

Two-factor authentication (2FA) adds a second verification step after the password. In this Python demonstration, a six-digit OTP is generated and displayed in the console for testing.

```
import secrets

correct_password = "Paytm@123"

password = input("Enter password: ")

if password == correct_password:
    otp = f"{secrets.randbelow(1000000):06d}"
    print("Your demo OTP is:", otp)  # Console display for this exercise

    entered_otp = input("Enter the 6-digit OTP: ")

    if entered_otp == otp:
        print("Login Successful")
    else:
        print("Invalid OTP. Login Failed")
else:
    print("Incorrect password. Login Failed")
```

Sample console run:

```
Enter password: Paytm@123
Your demo OTP is: 482731
Enter the 6-digit OTP: 482731
Login Successful
```

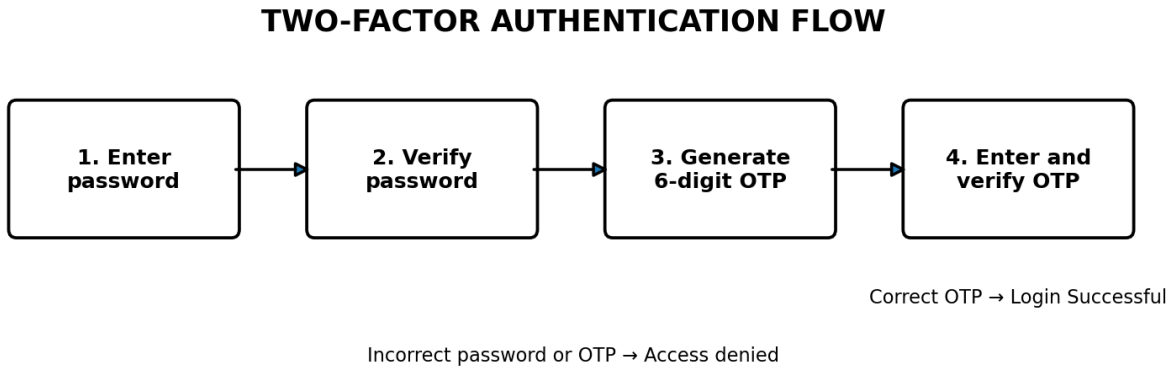


Figure 3: Steps in the two-factor authentication process.

Security note: Displaying an OTP in the console is only for classroom simulation. A real service sends the OTP through a verified channel, stores credentials securely, and applies expiry and attempt limits.

Result: Login is successful only when the password and generated OTP are both correct.

TASK: 4

Aim: Filter user objects to return only premium users who logged in within the last seven days, using 2024-06-10 as today.

Answer:

The following JavaScript function compares each user's lastLogin date with the seven-day period from 2024-06-03 through 2024-06-10. It includes both boundary dates.

```
function getRecentPremiumUsers(users) {
  const today = new Date("2024-06-10T00:00:00");
  const sevenDaysAgo = new Date("2024-06-03T00:00:00");

  return users.filter(user => {
    const loginDate = new Date(user.lastLogin + "T00:00:00");
    return user.isPremium === true &&
           loginDate >= sevenDaysAgo &&
           loginDate <= today;
  });
}

const users = [
  { username: "Aarav", isPremium: true,  lastLogin: "2024-06-09" },
  { username: "Diya",  isPremium: false, lastLogin: "2024-06-08" },
  { username: "Kabir", isPremium: true,  lastLogin: "2024-06-02" },
  { username: "Meera", isPremium: true,  lastLogin: "2024-06-03" },
  { username: "Riya",  isPremium: true,  lastLogin: "2024-06-10" }
];

console.log(getRecentPremiumUsers(users));
```

Expected result:

```
[
  { username: "Aarav", isPremium: true, lastLogin: "2024-06-09" },
  { username: "Meera", isPremium: true, lastLogin: "2024-06-03" },
  { username: "Riya",  isPremium: true, lastLogin: "2024-06-10" }
]
```

Explanation: Diya is excluded because she is not a premium user. Kabir is excluded because his last login was before 2024-06-03. The other three users meet both conditions.

Result: The function returns only premium users who logged in during the specified seven-day period.

TASK: 5

Aim: List five common access control mistakes in real-world applications and provide a prevention tip for each.

Answer:

Access control ensures that users can access only the data and functions permitted for their accounts. The following are five common mistakes that can affect applications such as Instagram or Flipkart.

Common mistake	What can go wrong	Fix / prevention tip
1. Broken access control	Mistake: The application does not properly check whether a user is allowed to open a page or perform an action.	Fix: Enforce permission checks on the server for every protected request.
2. Insecure Direct Object References (IDOR)	Mistake: A user changes an order, profile or account ID in a request and accesses another user's information.	Fix: Verify ownership or authorization for every requested object; never rely only on an ID being hard to guess.
3. Excessive user permissions	Mistake: A normal customer receives administrator-level permissions unnecessarily.	Fix: Follow the principle of least privilege and assign only the permissions needed for each role.
4. Missing authorization on APIs	Mistake: A hidden button is removed from the screen, but the related API endpoint still accepts unauthorized requests.	Fix: Check authorization on the server-side API, not only in the user interface.
5. Weak session management	Mistake: Sessions remain active for too long or are not invalidated after logout or password changes.	Fix: Use secure session cookies, suitable timeouts, session rotation and server-side invalidation.

Conclusion: Strong access control protects user accounts, private information and important application functions. Developers should validate permissions on the server, use least privilege and manage sessions securely.

Result: Five common access control mistakes and their prevention methods have been identified.